

Vibe Coding: Coding with Agentic AI

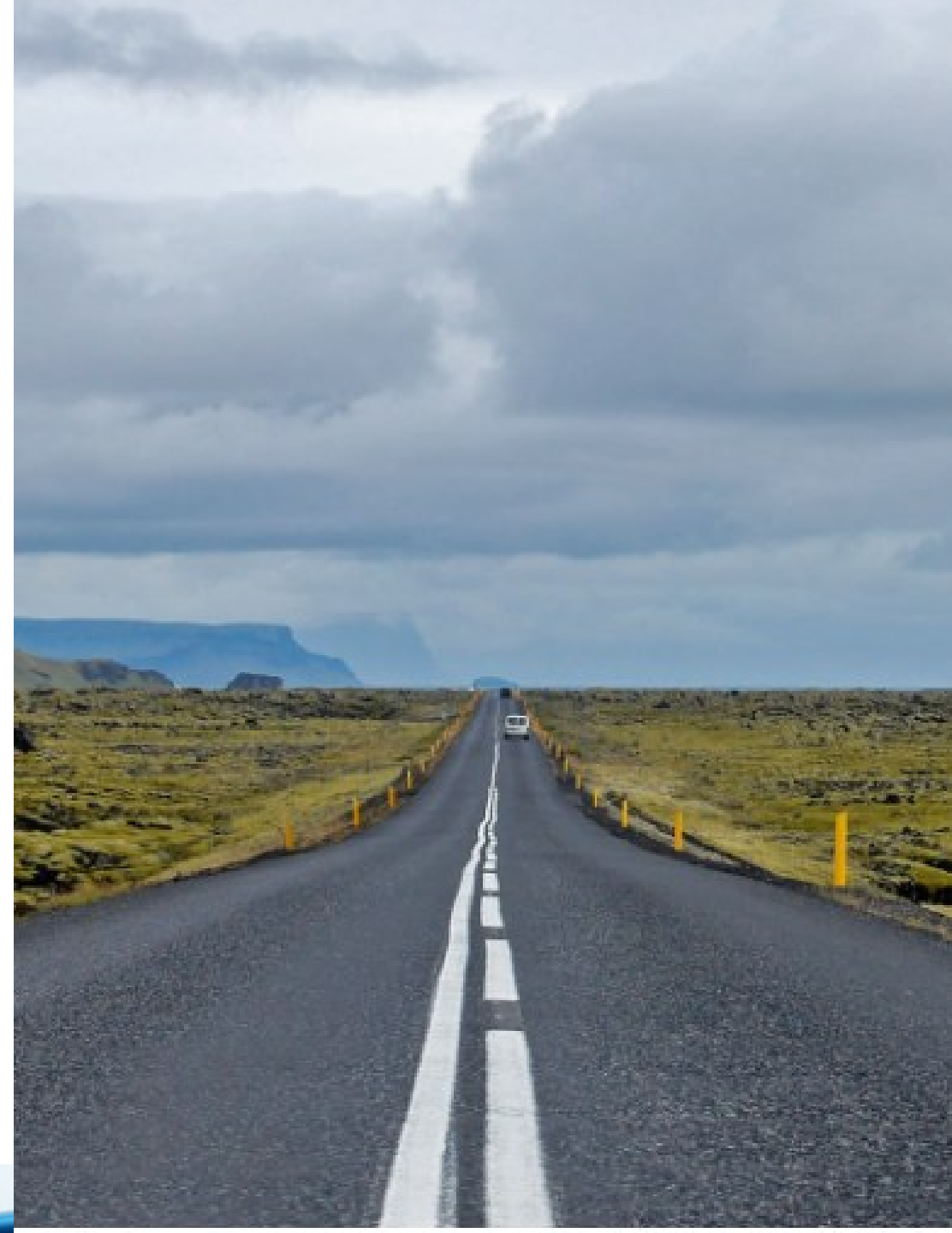
Kevin Lee, Nathan Lee



Proprietary and Confidential Information – Do Not Copy or Distribute

Agenda

- Introduction of Vibe-coding
- Vibe Coding with
 - ChatGPT.com
 - Claude.ai/new
 - <https://gemini.google.com/app>
- Demo: Vibe Coding with Cursor
- Vibe Coding with Codezen.ai in [CodeZen.ai® - CodeLabs](#)
- Demo: Vibe Coding with Personal, Local AI Agents



Where “Vibe-Coding” come from?

“fully giv[ing] in to the vibes, embrac[ing] exponentials, and forget[ting] that the code even exists.”

by Andrej Karpathy,
co-founder of OpenAI
on Feb, 2025



“There's a new kind of coding I call "vibe coding," where you fully give in to the vibes.”

-Andrej Karapathy

EDUCATION **NEXT** 

What is Vibe-Coding?

- Coding with AI
- Vibe : A flow between human logic and AI coding agents
- Programmers do
 - Prompt for the codes
 - Review the codes



Vibe Coding

- Vibe Coding with
 - ChatGPT.com
 - Claude.ai/new
 - <https://gemini.google.com/app>
- GitHub for HoT in
- Demo: Vibe Coding with Cursor
- Vibe Coding with Codezen.ai in [CodeZen.ai® - CodeLabs](#)
- Demo: Vibe Coding with Personal, Local AI Agents

ChatGPT: Codes Development – R

Prompt 1:

Write a **R script** that groups data by "TRTA" and "AVISIT" columns in "advs.xpt"

and calculates the mean of "AVAL" column for each group at PARAMCD = 'WEIGHT'

Prompt 2:

Write a **R script** that groups data by actual treatment and actual visit in "advs.xpt"

and calculates the mean for each group at weight.

Use ADaM CDISC variable.

Output :

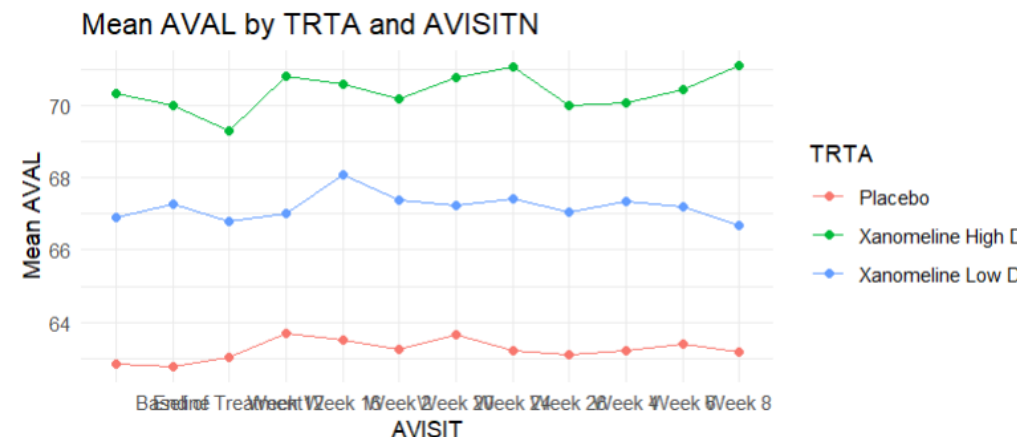
```
library(haven) # For reading XPT files
library(dplyr) # For data manipulation
```

```
# Read the ADaM dataset
advs <- read_xpt("advs.xpt")
```

```
# Process the data
result <- advs %>%
  filter(PARAMCD == "WEIGHT") %>% # Filter for PARAMCD = 'WEIGHT'
  group_by(TRTA, AVISIT) %>%      # Group by TRTA and AVISIT
  summarise(mean_AVAL = mean(AVAL, na.rm = TRUE)) %>%
  # Calculate mean of AVAL
```

```
ungroup()
```

```
# Print the result
print(result)
```



ChatGTP: Codes Development – SAS

Prompt :

Write a **SAS script** that groups data by “TRTA” and “AVISIT” columns in "adv.sxpt"

and calculates the mean of “AVAL” column for each group at PARAMCD =‘WEIGHT’

Output :

```
libname mydata xport 'path_to_adv.sxpt'; /* Replace with the actual path to adv.sxpt */
```

```
data weight_data;  
  set mydata.adv.sxpt;  
  if PARAMCD = 'WEIGHT'; /* Filter data for PARAMCD = 'WEIGHT' */  
run;
```

```
proc sql;  
  create table weight_means as  
  select TRTA, AVISIT,  
         mean(AVAL) as mean_weight  
  from weight_data  
  group by TRTA, AVISIT;  
quit;
```

```
proc print data=weight_means;  
run;
```


ChatGPT: Codes Development – Python

Prompt :

Write a **Python script** that groups data by “TRTA” and “AVISIT” columns in “adv.s.xpt”

and calculates the mean of “AVAL” column for each group at PARAMCD = ‘WEIGHT’

Output :

```
import pandas as pd
import xport

# Load the .xpt file using xport
with open('adv.s.xpt', 'rb') as f:
    data = xport.load(f)

# Convert the xport data to a pandas DataFrame
df = pd.DataFrame(data['ADVS'])

# Filter the data for PARAMCD = 'WEIGHT'
df_weight = df[df['PARAMCD'] == 'WEIGHT']

# Group by TRTA and AVISIT, then calculate the mean of AVAL
grouped_data = df_weight.groupby(['TRTA', 'AVISIT'])['AVAL'].mean().reset_index()

# Output the grouped data
print(grouped_data)
```


Data Visualization

- Prompt :
Create line graph of “Patient Data” in R programming

“Patient Data”:

```
{ 'patient' : '01-001', 'visit 1' : 100, 'visit 2' : 105, 'visit 3' : 110},  
{ 'patient' : '01-002', 'visit 1' : 104, 'visit 2' : 95, 'visit 3' : 90},  
{ 'patient' : '02-001', 'visit 1' : 99, 'visit 2' : 103, 'visit 3' : 105},  
{ 'patient' : '02-002', 'visit 1' : 110, 'visit 2' : 105, 'visit 3' : 115}
```

Data Visualization

- Prompt :
Create SAS codes based on below SDTM spec for DM domain:

#spec:

Variable_Label	Type	Codelist	Derivation
Study Identifier	Char		DEMO.STUDYID
Domain Abbreviation	Char		DM
Unique Subject Identifier	Char		DEMO.STUDYID + '-' + DEMO.SITE_ID + '-' + DEMO.SUBJID
Subject Identifier for the Study	Char		DEMO.SUBJID
Subject Reference Start Date/Time	Char		first EXPO.EXDTC per subject
Subject Reference End Date/Time	Char		last EXPO.EXDTC per subject if available
First Date/Time of Exposure	Char		first EXPO.EXDTC per subject
Last Date/Time of Exposure	Char		last EXPO.EXDTC per subject
Informed Consent	Char		select DISP.DSDTC where DISP.DISP = 'CONSENT' per subject
Date/Time of End of Participation	Char		select latest date across (EXPO.EXDTC, DISP.DSDTC) per subject
Date/Time of Death	Char		select DISP.DSDTC where DISP.DISP = 'DEATH' per subject
Subject Death Flag	Char		derive 'Y' IF DISP.DSDTC is not missing where DISP.DISP = 'DEATH' for subject
Study Site Identifier	Char		DEMO.SITE_ID
Date / Time of Birth	Char		DEMO.BIRTHDTC
Age	Num		year of numeric value of (RFICDTC - BRTHDTC + 1)
Age Units	Char		YEARS
Sex	Char		DEMO.SEXC
Race	Char		DEMO.RACEC
Planned Arm Code	Char	ARMCD	RAND.TRT_P_CD
Description of Planned Arm	Char		RAND.TRT_P
Actual Arm Code	Char		If RAND.TRT_A = 'Control' then 'C'. If RAND.TRT_A='Study Drug' then 'SD'
Description of Actual Arm	Char		RAND.TRT_A
Reason Arm and/or Actual Arm is Null	Char		RAND.RS_NULL
Description of Unplanned Actual Arm	Char		RAND.RS_DESC
Country	Char		USA



Demo: Vibe Coding with Cursor

Vibe Coding with Coding Agent

<https://167.172.112.209/login>
codezentemp@clinvia.com
codezen123@

Demo: Vibe Coding with Local, Personal AI Agents

AI Agents

- System that can interact with its environment, use the tools, and **autonomously** perform tasks and make decisions to achieve the goals.

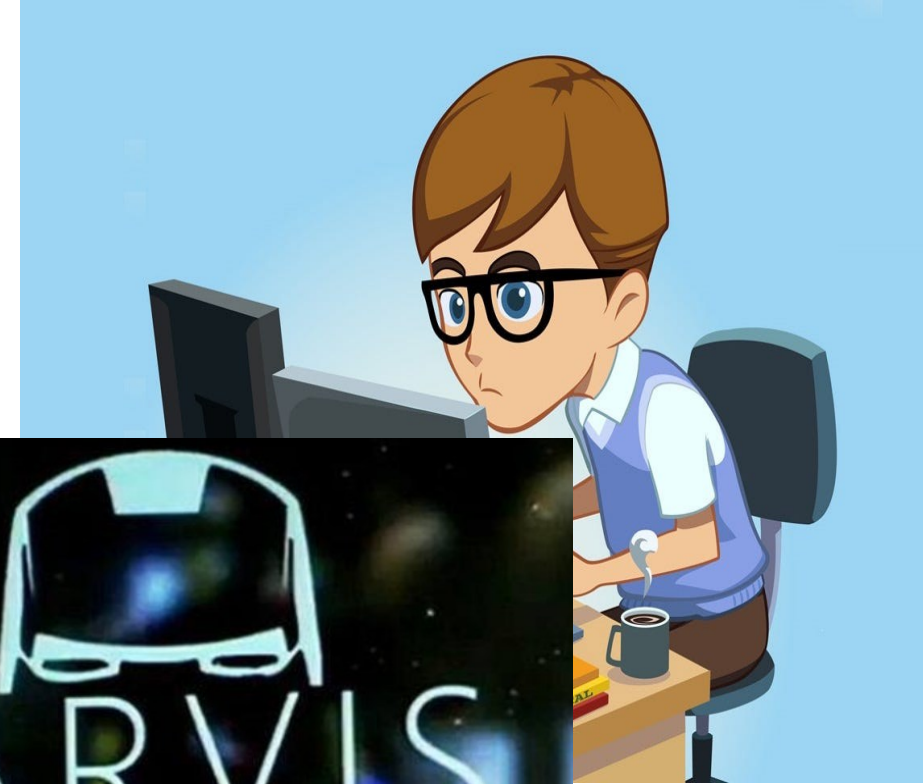
- Main Components
 - LLM (e.g., ChatGPT5.2, Claude4.5)
 - Prompts: (e.g., instruction for AI Agent –
'You are stat programmer.
Use the input(e.g., SDTM spec) and generate SAS output.

Follow below steps:

1. Generate SAS codes based on input
2. Execute generated SAS codes using SAS Studio.
3. If there are error messages, debug SAS codes.
4. Rerun SAS codes to create the output.

Do Step 2 to 4 until there are no error message. '

- Tools (e.g., SAS Studio, R Studio, Python)
- Memory : short-term /long-term memory/ RAG (e.g., previous SAS codes)
- Skill: specific executable capabilities and domain-specific **SOPs or WGs** that AI Agents can follow and execute (e.g., SDTM/ADaM/TFL codes development, R Shiny)



Personal AI Agents in SCE

What It Is

A secure, local multi-agent AI system that automates clinical programming with built-in compliance and validation.

How It Works

- AI Planning
- Human Approved
- AI Execution
- Summarization by AI

AI Agents execute SOP-driven workflows with guardrails and full traceability (history, logs, audit-trail).

Compliance by Design

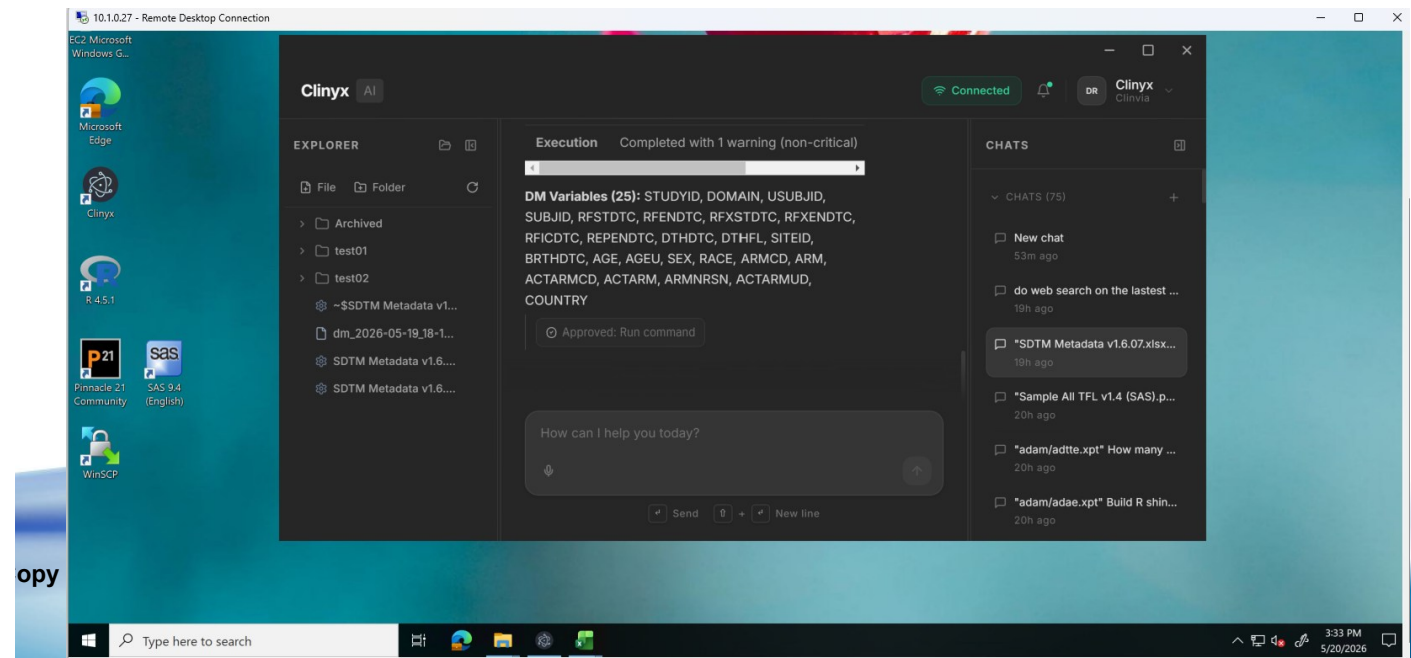
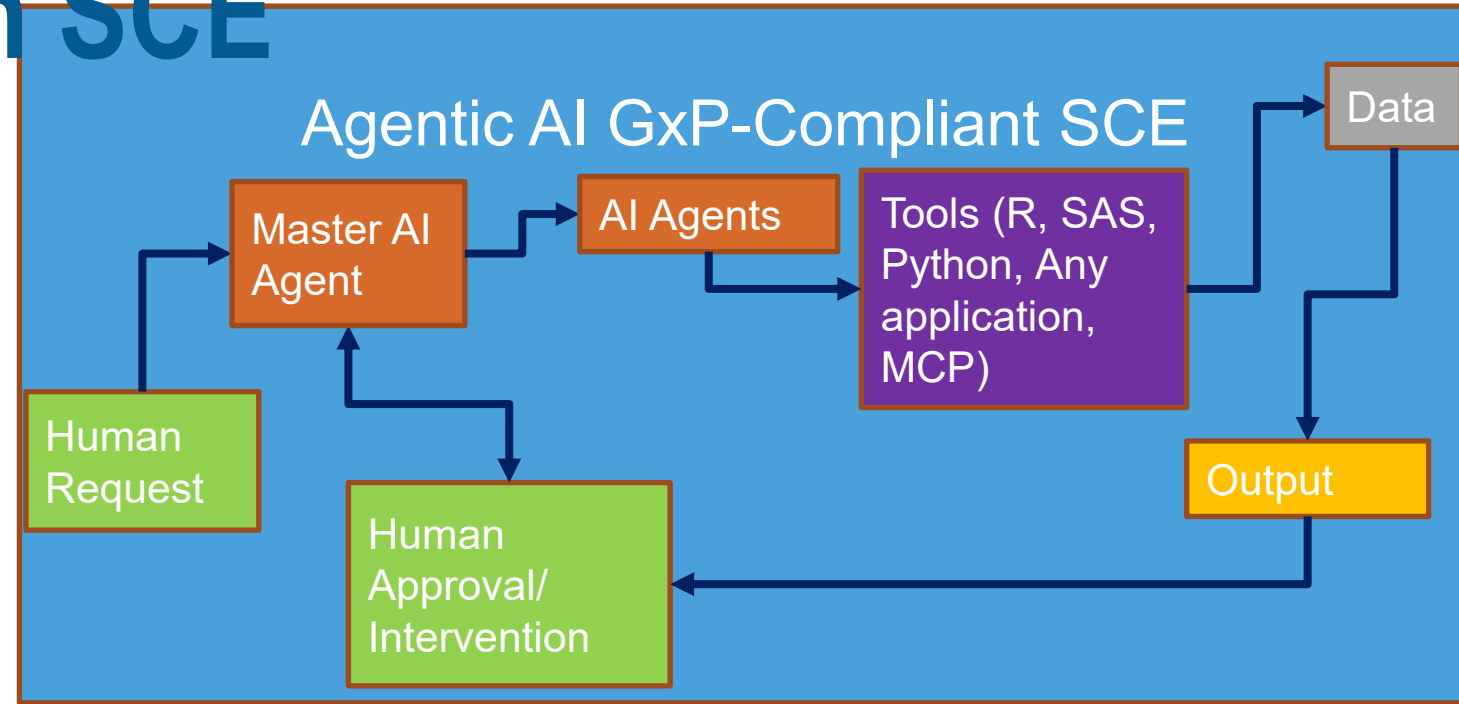
GxP-aligned (IQ/OQ/PQ ready), supports 21 CFR Part 11, and runs in GxP compliant SCE with complete audit trails, data security and access control.

Value & Use Cases

- Ease to use
- Conversational
- Accelerates Biometrics Tasks such as SDTM/ADaM, TLFs, and documentation with standardized, QC-validated, inspection-ready outputs while ensuring data security.

Differentiator

Personal, validated, auditable, SOP-driven Agentic AI purpose-built for regulatory clinical development. 20 to 50% production increased.



ANY QUESTIONS

Kevin Lee at kevin@clinvia.com

LinkedIn: <https://www.linkedin.com/in/HelloKevinLee>

Natha Lee at nathan@clinvia.com

LinkedIn: <https://www.linkedin.com/in/nathanlee-ds/>

